YENEPOYA UNIVERSITY

Project – Low Level Design (LLD)

on

Natural Language Interface for E-Commerce Infrastructure

(ShopEase E-Commerce Chatbot)

Student Name: Muhammed Yasar KMC

Course: BCA

Industry Mentor: Sumit Kumar Shukla

Guided by: Sindhu

Reg no : 23BCAICD086

Project code : PRJN26-093

Table of Contents

1. Introduction 3

1.1 Purpose of this Document 3

1.2 Scope 3

2. Module Design 4

2.1 chatbot.py — Detailed Design 4

2.2 responses.py — Detailed Design 4

2.3 utils.py — Detailed Design 5

2.4 index.html — Web Module Design 5

3. Function Specifications 6

3.1 run_chatbot() 6

3.2 get_response(user_input) 6

3.3 display_banner() 7

3.4 display_help() 7

3.5 format_bot_reply(message) 7

4. Algorithm Design 8

4.1 Keyword Matching Algorithm 8

4.2 Input Normalization 8

5. Data Structures 9

6. Error Handling 9

7. Test Cases 10

8. Limitations and Future Scope 10

1. Introduction

The Low Level Design (LLD) document provides a detailed technical description of the internal workings of the Natural Language Interface for E-Commerce Infrastructure (ShopEase Chatbot). While the HLD document describes what the system does and how components connect, this LLD document describes exactly how each component is implemented — including function-by-function breakdowns, pseudocode, algorithms, data structures, error handling, and test cases.

1.1 Purpose of this Document

The purpose of this LLD document is to:

- Describe every Python function in detail — its inputs, outputs, and internal logic
- Explain the keyword matching algorithm step by step with a worked example
- Define all data structures used in the project
- Document error handling and edge cases
- Provide test cases that verify correct chatbot behaviour
- Serve as a technical reference for anyone maintaining or extending the project

1.2 Scope

This LLD covers the internal design of all 3 Python modules (chatbot.py, responses.py, utils.py) and the JavaScript logic inside index.html. It does not cover deployment procedures, installation steps, or external API integration — those are outside the scope of this version of the project.

2. Module Design

The project is organized into 4 files. Each file has a single clearly defined responsibility, following the principle of Separation of Concerns.

2.1 chatbot.py — Detailed Design

This is the entry point of the entire application. It contains one main function — run_chatbot() — and is the only file that should be executed directly.

Property	Detail
File Name	chatbot.py
Imports	from responses import get_response
Imports	from utils import display_banner, display_help, format_bot_reply
Main Function	run_chatbot()
Loop Type	while True — infinite loop, broken only when user types exit
Input Method	Python built-in input() function
Exit Keywords	exit, quit, bye, goodbye
Help Keywords	help, commands, menu, ?

2.2 responses.py — Detailed Design

This module is the brain of the chatbot. It contains the entire knowledge base and the matching logic.

Property	Detail
File Name	responses.py
Data Structure	RESPONSES — Python list of dictionaries
Number of Categories	7 (order status, cancel, return, payment, shipping, offers, support)
Fallback Variable	FALLBACK — string returned when no keyword matches
Main Function	get_response(user_input: str) -> str
Matching Method	Python 'in' operator — checks if keyword is a substring of input
Normalization Applied	user_input.lower().strip() before any matching begins

2.3 utils.py — Detailed Design

This module handles all display and formatting tasks. It uses only Python's built-in print() function and the standard time module — no third-party libraries.

Function	Internal Logic
display_banner()	Uses print() with '=' characters to create a bordered welcome header
display_help()	Iterates a list of (command, description) tuples. Uses f-string with {cmd:<18} for left-aligned column formatting
format_bot_reply(message)	Calls time.sleep(0.3) for a natural delay. Splits message on newline character. Prints each line with 2-space indentation inside a '-' bordered box

2.4 index.html — Web Module Design

The web version is a single self-contained HTML file with three internal sections. No external files, frameworks, or servers are needed.

Section	Purpose
<style> block	CSS — defines layout, colors, WhatsApp-style chat bubbles, typing animation, quick buttons
<body> HTML	Structure — chat window container, header bar, messages area, input bar, quick buttons
<script> block	JavaScript — getResponse(), sendMessage(), addMessage(), showTyping(), hideTyping(), sendQuick()

3. Function Specifications

3.1 run_chatbot()

Attribute	Detail
File	chatbot.py
Parameters	None
Return Value	None — runs until user exits
Description	Initializes the chatbot, displays the welcome banner, and manages the continuous input loop until the user types an exit keyword

Pseudocode:

```
FUNCTION run_chatbot():

  CALL display_banner()

  PRINT 'Type help for commands. Type exit to quit.'

  LOOP forever:

    GET user_input from keyboard using input()

    IF user_input is empty:

      PRINT 'Please type something'

      CONTINUE to next loop iteration

    IF user_input is in [exit, quit, bye]:
```

```
      PRINT 'Goodbye'
      BREAK out of loop
    IF user_input is in [help, ?]:
      CALL display_help()
      CONTINUE to next loop iteration
    response = CALL get_response(user_input)
    CALL format_bot_reply(response)
  END LOOP
```

3.2 get_response(user_input)

Attribute	Detail
File	responses.py
Parameter	user_input (str) — raw text entered by the user
Return Value	str — the matched response string, or FALLBACK if nothing matches
Description	Normalizes the input, then searches the RESPONSES list for any matching keyword using the 'in' operator. Returns the first match found.

Pseudocode:

```
FUNCTION get_response(user_input):
  normalized = user_input.lower().strip()
  FOR each entry in RESPONSES list:
    FOR each keyword in entry['keywords']:
      IF keyword is found inside normalized:
```

```
        RETURN entry['response']  ← exit immediately

  RETURN FALLBACK  ← only reached if no match found
```

3.3 display_banner()

Attribute	Detail
File	utils.py
Parameters	None
Return Value	None
Description	Prints a formatted ASCII welcome banner to the terminal using multiple print() calls with '=' characters as borders. Called once when the chatbot starts.

3.4 display_help()

Attribute	Detail
File	utils.py
Parameters	None
Return Value	None
Description	Iterates a hardcoded list of (command, description) tuples. Uses Python f-string formatting with left-align padding {cmd:<18} to create a neat two-column layout. Triggered when user types help.

3.5 format_bot_reply(message)

Attribute	Detail
File	utils.py
Parameter	message (str) — the response string to display
Return Value	None
Description	Calls time.sleep(0.3) to simulate a natural thinking delay. Splits the message string on the newline character. Prints each resulting line with 2-space indentation, surrounded by a '-' character border for visual clarity.

4. Algorithm Design

4.1 Keyword Matching Algorithm

The core algorithm that powers the entire chatbot is a Sequential Linear Search through the RESPONSES list:

Property	Value
Algorithm Type	Sequential Linear Search
Time Complexity	O(R x K) where R = entries (7), K = keywords per entry (~5)
Space Complexity	O(1) — no additional memory allocated during the search
Match Strategy	First match wins — function returns immediately on first keyword found
Case Sensitivity	Case-insensitive — input is normalized to lowercase before search begins
Worst Case Checks	35 comparisons (7 x 5) — completes in under 1 millisecond

Step-by-step worked example for input: 'Can I track my order please?'

Step	Check Performed	Result
1	Normalize input	'can i track my order please?'
2	Entry 0, keyword: 'order status' in text?	Not found — continue
3	Entry 0, keyword: 'track order' in text?	FOUND — return ORDER STATUS response
4	(no further checks needed)	Function exits immediately

4.2 Input Normalization

Before any matching is performed, user input is processed through two Python string methods:

Method	Purpose	Example
.lower()	Converts all characters to lowercase	'SHOW STATUS' becomes 'show status'
.strip()	Removes leading and trailing whitespace	' show status ' becomes 'show status'

These two operations ensure that typing in capital letters or with accidental spaces still produces a correct match — making the chatbot more forgiving and natural to use.

5. Data Structures

The project uses two data structures, both defined inside responses.py:

5.1 RESPONSES — List of Dictionaries

Property	Detail	
Variable Name	RESPONSES	
Type	Python list containing dictionaries	
Total Entries	7 — one per e-commerce support category	
Key: keywords	List of strings — triggers for this response	
Key: response	String (multi-line using \n) — the pre-written reply	
Access Pattern	Sequential read only — never modified during runtime	
Memory Size	Approximately 2KB — negligible	

5.2 FALLBACK — String Constant

Property	Detail
Variable Name	FALLBACK
Type	Python string (str)
Purpose	Returned by get_response() when no keyword from any RESPONSES entry matches
Content	A polite message suggesting the user try known commands or type 'help'

6. Error Handling

Error / Edge Case	How It Is Handled
User presses Enter with no text	Detected by 'if not user_input' check — bot asks user to type something and continues the loop
User types unrecognized text	No keyword match is found — FALLBACK response is returned gracefully
User presses Ctrl+C (interrupt)	Caught by try/except KeyboardInterrupt — prints goodbye message and exits cleanly without a crash
User types in ALL CAPS	Handled by .lower() normalization — 'SHOW STATUS' matches same as 'show status'
User types with extra spaces	Handled by .strip() normalization — ' show status ' matches correctly
Web version — empty send	JavaScript checks if userInput is empty before processing — ignores blank sends
Web version — XSS attempt	JavaScript uses textContent instead of innerHTML — malicious script tags are treated as plain text and never executed

7. Test Cases

The following test cases verify that the chatbot behaves correctly for all expected and unexpected inputs:

Test Input	Expected Output	Pass / Fail
show status	Order status with tracking ID	Pass
SHOW STATUS (all caps)	Same order status reply	Pass
show status (extra spaces)	Same order status reply	Pass
Can I track my order?	Order status (keyword: track order)	Pass
cancel	Cancel order steps	Pass
I want a refund	Returns and refunds reply	Pass
payment options	Payment methods list	Pass
any offers available?	Current discounts and coupons	Pass
how long does shipping take?	Shipping info (keyword: how long)	Pass
I need support	Customer support contacts	Pass
hello xyz random text	FALLBACK — unrecognized message	Pass
(empty — just Enter pressed)	'Please type something' prompt	Pass
help	Full command reference table	Pass
exit	Goodbye message and program closes	Pass

8. Limitations and Future Scope

8.1 Current Limitations

- No context memory — each message is processed independently with no conversation history
- Exact substring matching only — misspellings like 'ordr status' will not match any keyword
- Hardcoded responses — adding new topics requires editing the Python source file directly
- No real data — order status, offers, and tracking info are all simulated, not live
- Single user only — no support for multiple simultaneous users in the terminal version
- No voice input — text only interface

8.2 Future Improvements

Improvement	How It Could Be Implemented
Fuzzy matching for typos	Use Python's built-in difflib.get_close_matches() to match similar words
Conversation memory	Maintain a conversation_history = [] list to store previous exchanges
Live database connection	Connect to SQLite or MySQL for real order and product data
User login system	Add a simple username and session tracking mechanism
Voice input support	Integrate Python's speech_recognition library for spoken queries
Deploy online	Host index.html on GitHub Pages or Netlify for public internet access
Admin panel	Build a simple web form to add new keywords and responses without editing code